

Claude For Microsoft 365 Install Prompt Library

Comprehensive Translation, Config Coverage, and Dependency Map

financial-services-plugins repository

Generated on 2026-05-12 10:03

Contents

1	Repository Structure Overview	2
1.1	Inventory Summary	2
1.2	Folder Tree	2
2	Prompt Dependency Structure	3
3	Translated Prompt Corpus	4
3.1	Directory: .claude-plugin	4
3.2	Plugin Manifest: .claude-plugin	4
3.3	Directory: commands	4
3.4	Command: bootstrap.md	4
3.5	Command: consent.md	10
3.6	Command: debug.md	12
3.7	Command: manifest.md	15
3.8	Command: setup.md	20
3.9	Command: update-user-attrs.md	27
3.10	Directory: examples	30
3.11	Script File: examples/python-bootstrap/app.py	30
3.12	Script File: examples/python-bootstrap/config.py	33
3.13	Script File: examples/python-bootstrap/get_tenant_id.py	35
3.14	Script File: examples/python-bootstrap/mint_dev_token.py	36
3.15	File: examples/python-bootstrap/README.md	38
3.16	File: examples/python-bootstrap/requirements.txt	39
3.17	Directory: README.md	39

3.18 File: README.md	39
3.19 Directory: scripts	40
3.20 Script File: scripts/build-manifest.mjs	40

1 Repository Structure Overview

1.1 Inventory Summary

Metric	Count
Total included files	15
Markdown or markdown-like files	8
JSON config files	1
YAML manifest files	0
Scripts	5
Other text files	1
Commands	6
Skills	0
Plugin manifests	1
Managed-agent manifests	0

1.2 Folder Tree

Root directory: `claude-for-msft-365-install`

- `.claude-plugin`
- `commands`
- `examples`
- `README.md`
- `scripts`
- `.claude-plugin/plugin.json`
- `commands/bootstrap.md`
- `commands/consent.md`
- `commands/debug.md`
- `commands/manifest.md`
- `commands/setup.md`
- `commands/update-user-attrs.md`
- `examples/python-bootstrap`
- `scripts/build-manifest.mjs`
- `examples/python-bootstrap/app.py`
- `examples/python-bootstrap/config.py`
- `examples/python-bootstrap/get_tenant_id.py`
- `examples/python-bootstrap/mint_dev_token.py`

- `examples/python-bootstrap/README.md`
- `examples/python-bootstrap/requirements.txt`

2 Prompt Dependency Structure

3 Translated Prompt Corpus

This section translates the prompt, manifest, configuration, and script files under `claude-for-msft-365-install` into a print-oriented format that preserves structure while improving readability.

3.1 Directory: `.claude-plugin`

3.2 Plugin Manifest: `.claude-plugin`

JSON Summary

Key	Type	Summary
name	str	claude-for-msft-365-install
description	str	Provision direct cloud access (Vertex AI, Bedrock, or LLM gateway) for the Claude Office add-in. Generates the customized add-in manifest, walks through Azure admin consent, and writes per-user config via Microsoft Graph extension attributes.
version	str	0.1.2
author	dict	object with 2 key(s)

Structured JSON Content

- {
- “name”: “claude-for-msft-365-install”,
- “description”: “Provision direct cloud access (Vertex AI, Bedrock, or LLM gateway) for the Claude Office add-in. Generates the customized add-in manifest, walks through Azure admin consent, and writes per-user config via Microsoft Graph extension attributes.”,
- “version”: “0.1.2”,
- “author”: {
- “name”: “Anthropic”,
- “email”: “support@anthropic.com”
- }
- }

3.3 Directory: `commands`

3.4 Command: `bootstrap.md`

File Metadata

Field	Value
description	Build the bootstrap endpoint — per-user MCP servers, skills, dynamic config

Bootstrap endpoint

You host an HTTPS GET handler. The add-in calls it at startup with the user's Entra token, you return per-user JSON, the response overrides manifest and extension attrs for that user. This is how you push structured config — *mcp_servers*, *skills* — that flat string attrs can't carry.

Ask first

Figure out which mode you're in before walking the spec:

- **Just want to understand it?** Answer from the sections below. Common questions: what's the response shape, how does `{{...}}` work, why is CORS biting me.
- **Building one?** Ask: new handler or editing an existing one? Lambda, Cloud Function, Express, Python, something else? Then jump to [Scaffolding](#) — the sections in between are the contract you're coding against.

This vs extension attrs

Both deliver per-user config. Pick by what you're carrying.

	Extension attrs	Bootstrap endpoint
You write	<i>az rest PATCH</i> per user	An HTTPS service
Carries	Flat strings, ≤256 chars	Any JSON — arrays, nested, base64
Good for	Token rotation, region override	<i>mcp_servers</i> , <i>skills</i> , anything structured
Refresh	Token cache, ~1hr lag	<i>bootstrap_expires_at</i> , you control it
Auth	Entra token claims (passive)	You validate the JWT (active)

If you only need to swap *gateway_token* per user, attrs are less work. The moment you want a Linear MCP server for one team and a Jira one for another, you're here.

Template interpolation

Any string value can contain `{{key}}`. The add-in substitutes against the **merged config chain** — manifest params, then extension attrs, then this response, each layer overriding the last. You don't echo a value back just so a template can see it; if *gateway_token* is already in the manifest or an attr, `{{gateway_token}}` resolves.

Two phases, because the request has to happen before the response exists:

1. ***bootstrap_url* itself** resolves against manifest + attrs only. So the manifest can carry *bootstrap_url=https://config.internal/bootstrap?project={{gcp_project_id}}* and you run one endpoint that branches on a query param instead of stamping per-team URLs into attrs.

2. **Response fields** resolve against the full merge — manifest + attrs + whatever this response just returned. An *mcp_servers* entry can reference a *gateway_token* that lives three lines up in the same JSON.

Unresolved `{{key}}` (typo, key never set anywhere) is left as-is in the string — no error, no empty-substitution. If an MCP server isn't connecting, check the URL the add-in actually constructed.

CORS — every URL needs it

The add-in is a browser. Every fetch — *bootstrap_url*, every *mcp_servers[].url*, every *skills[].url* — happens browser-side from inside the Office taskpane. Without *Access-Control-Allow-Origin: https://pivot.claude.ai* on the response, the browser blocks it before the add-in sees a byte. The server returns 200, the add-in gets nothing, and nothing in the add-in's logs tells you why. This is the most common “it's not working” cause.

URL	Where CORS lives
<i>bootstrap_url</i>	Your handler's response headers. Behind API Gateway / Cloud Functions, also configure the <i>OPTIONS</i> preflight — the browser sends one before any request with custom headers. See the recommended preflight response below.
<i>mcp_servers[].url</i>	The MCP server itself. Public ones (Linear, Atlassian) already allow it. Internal ones almost certainly don't until you add it.
<i>skills[].url</i>	The bucket, not the URL. Presigned URLs auth the request — they don't grant CORS. S3 needs a bucket CORS config, GCS needs <i>gsutil cors set</i> , Azure needs blob service CORS rules.
<i>otlp_endpoint</i>	Your OTEL collector's HTTP receiver. Most collectors default to same-origin only — set <i>cors.allowed_origins</i> on the OTLP/ HTTP receiver.

For *bootstrap_url*, the recommended preflight response is:

Structured Template

- Access-Control-Allow-Origin: <https://pivot.claude.ai>
- Access-Control-Allow-Methods: GET
- Access-Control-Allow-Headers: Authorization, X-Claude-User-Agent, *

Allowing * for request headers is safe here — security comes from the Entra token, not header filtering — and keeps preflights working if the add-in adds headers in future. Keep *Allow-Origin* pinned to <https://pivot.claude.ai>.

The presigned-URL one bites hardest because *curl* works (curl ignores CORS), the signature is valid, the object exists, and the skill still doesn't load. Set bucket CORS once:

Structured Template

- // S3 — `aws s3api put-bucket-cors --bucket --cors-configuration file://cors.json`
- { “CORSRules”: [{ “AllowedOrigins”: [“<https://pivot.claude.ai>”], “AllowedMethods”: [“GET”], “Allowed-Headers”: [“*”] }] }

Structured Template

- # GCS — gsutil cors set cors.json gs://
- [{"origin": ["https://pivot.claude.ai"], "method": ["GET"], "responseHeader": ["*"]}]

Structured Template

- # Azure — az storage cors add --services b --methods GET --origins https://pivot.claude.ai --allowed-headers '*' --account-name

If you're debugging a CORS failure: open the browser devtools inside the taskpane (right-click → Inspect on Windows, or attach via Safari's Develop menu on Mac), look for the request in the Network tab. A CORS block shows as a failed request with no response body and a console error naming the origin.

Request

Structured Template

- GET # after interpolation
- Authorization: Bearer # only if entra_sso=1 in manifest
- X-Claude-User-Agent: claude-/# always sent

X-Claude-User-Agent identifies which Office host the add-in is running in. *'is one of word, excel, or powerpoint; is the add-in build (e.g. claude-excel/1.4.2)'*. Use it to return different skills or MCP servers per Office product, or to gate the add-in to specific hosts for a user.

Without *entra_sso=1* there's no Authorization header — the request is anonymous from the add-in's side. That's fine if the endpoint sits behind network isolation, mTLS, or another auth layer the add-in doesn't see.

With *entra_sso=1*, validate the JWT before trusting it:

Claim	Check
<i>aud</i>	<i>c2995f31-11e7-4882-b7a7-ef9def0a0266</i> — the add-in's default app ID, or your own app's GUID if you set <i>graph_client_id</i> in the manifest . Anything else means the token wasn't minted for this.
<i>iss</i>	<i>https://login.microsoftonline.com/v2.0</i> — your tenant. Reject other tenants.
<i>exp</i>	Not expired. Libraries handle this; don't hand-roll it.
<i>oid</i>	The user's stable object ID. This is your lookup key — email (<i>upn/preferred_username</i>) can change, <i>oid</i> doesn't.

If you set *entra_scope* in the [manifest](#), the Bearer is an **access token**, not an ID token. Validate *aud* = your API's Application ID URI (*api://*, not the client GUID) and check *scp* contains the scope you defined. *iss*, *exp*, *oid*, and signature verification are the same.

Signature verification needs Microsoft's JWKS (<https://login.microsoftonline.com/discovery/v2.0/keys>). Use a library — *jose* (Node), *PyJWT + cryptography* (Python), *Microsoft.IdentityModel.Tokens* (.NET). Hand-rolled JWT verification is where security bugs live.

Response

200 OK, Content-Type: application/json, CORS header per [above](#).

The body is a flat object. Every field is optional — return only what differs for this user. Unknown keys are ignored, so you can add fields the current add-in version doesn't read yet and they'll light up when it ships.

Provider keys

Any of the cloud config keys from the [manifest](#) table — *gateway_url*, *gateway_token*, *aws_role_arn*, *gcp_region*, etc. Same names, same meanings, just per-user.

If you return *gateway_api_format*: “vertex”, also return *gcp_project_id* and *gcp_region* (or set them at a lower layer) — they're path segments in the Vertex *:rawPredict* URL the add-in constructs. “bedrock” needs no extras.

Telemetry

Structured Template

- “otlp_endpoint”: “https://otel-collector.your-domain.com”,
- “otlp_headers”: “Authorization=Bearer {{gateway_token}}”,
- “otlp_resource_attributes”: “team.name={{team}},deployment.environment=prod”

otlp_endpoint is the base HTTPS URL of an OpenTelemetry collector you operate; the add-in appends */v1/traces* and posts OTLP/HTTP. *otlp_headers* uses the standard *key1=value1,key2=value2* format and interpolates like any other value. *otlp_resource_attributes* uses the same format (matching the standard *OTEL_RESOURCE_ATTRIBUTES* variable) and is merged into the OpenTelemetry Resource on every span — use it when your collector requires specific resource attributes for routing or attribution. The collector must allow CORS from the add-in origin — see [above](#).

inference_headers

Structured Template

- “inference_headers”: { “x-application-id”: “app123” }

Extra HTTP headers attached to every request the add-in sends to your gateway (*gateway_url*) — typically accounting tags the gateway uses for cost allocation. Applies only to gateway deployments; direct cloud connections ignore it. The add-in treats them as opaque pass-through; *Authorization*, *x-api-key*, *Content-Type*, *Host*, *Content-Length*, *User-Agent*, *Cookie*, and any *anthropic-** / *x-amz-** / *x-goog-** header are reserved and dropped.

mcp_servers

Array of MCP servers the add-in connects to for this user.

Structured Template

- “mcp_servers”: [
 - { “url”: “https://mcp.linear.app/sse”, “label”: “Linear” },
 - {
 - “url”: “https://internal.yourcompany.com/mcp/risk”,

- “label”: “Risk Dashboard”,
- “headers”: { “Authorization”: “Bearer {{gateway_token}}” }
- }
-]

Field	
<i>url</i>	MCP server endpoint. Interpolated.
<i>label</i>	Display name in the add-in UI.
<i>headers</i>	Optional. Sent on every request to that server. Values interpolated — this is how you thread a per-user token through without the endpoint ever seeing it.

skills

Array of skills loaded for this user. Each is either inlined as base64 or fetched from a URL — set one or the other.

Structured Template

- “skills”: [
- {
- “name”: “deal-memo”,
- “description”: “Draft a deal memo from a term sheet”,
- “url”: “https://yourbucket.s3.amazonaws.com/skills/deal-memo.zip?X-Amz-...”
- },
- {
- “name”: “compliance-check”,
- “content”: “IyBDb2lwbGlhbmNIIGNoZWNRcGpSZXZpZXcgdGhlIGRvY3VtZW50IGZvci4uLg==”
- }
-]

Field	
<i>name</i>	Skill identifier. Interpolated.
<i>description</i>	Optional. Shown in the skill picker.
<i>content</i>	Base64 bytes. Either a zip (full skill bundle with <i>SKILL.md</i> + assets) or the raw <i>SKILL.md</i> text — the add-in sniffs which on decode.
<i>url</i>	Presigned URL (S3, GCS, Azure SAS). Bare GET, no auth headers added — bake auth into the signature. Response body sniffed the same way as <i>content</i> . Interpolated.

Inline *content* is simplest for small text-only skills. Use *url* once you’re shipping zips with images or the base64 starts bloating the response.

disabled_features

JSON array of feature slugs to lock for this user. Same vocabulary as the [manifest key](#) — bootstrap is the per-user layer.

Structured Template

- “disabled_features”: [“skills.authoring”]

bootstrap_expires_at

Epoch timestamp (seconds or milliseconds — auto-detected) for when this config goes stale. The add-in re-fetches before expiry. Omit and the config lives until the taskpane reloads.

Set this when you’re vending short-lived tokens. Don’t set it as a keepalive — if nothing in the response expires, the refetch is wasted.

Scaffolding a handler

A runnable Python/FastAPI reference with RBAC for *skills* and *mcp_servers* lives at [examples/python-bootstrap/](#) — point them there if they want something to copy.

When they want one built, write it for them. The contract above is what you’re coding against. Get these right:

JWT validation is the security boundary. Verify signature against Microsoft’s JWKS, check *aud* and *iss* exactly, pull *oid* for the user lookup. A handler that skips this and trusts *preferred_username* from an unverified token is an open endpoint with extra steps.

CORS on every URL you return, not just the handler — see the [CORS section](#). Easy to ship a working handler that returns presigned skill URLs from a bucket with no CORS config, and the skills never load.

User lookup is their business logic. Leave a clear *// TODO: look up config for oid* where the real work goes — DynamoDB, Postgres, a YAML file, whatever they have. Don’t guess; ask what their source of truth is.

Return sparse. Only the keys that differ from manifest defaults. An empty *{}* is a valid response — means “this user gets the org-wide config.”

Ask before writing: Lambda + API Gateway, Cloud Function, plain Express, something else? And where does per-user config live — inline in the handler (fine for a pilot), or read from a store?

3.5 Command: consent.md

File Metadata

Field	Value
description	Azure admin consent URLs — one-time tenant approval for Entra SSO and Outlook Graph access

Azure admin consent

Only needed when *entra_sso=1* in the manifest. Gateway and Vertex setups with org-wide config don't use Entra and can skip this. If you set *graph_client_id* (your own Entra app), this page doesn't apply either — you manage consent on your app directly.

One-time per tenant. A Global Admin opens this URL, clicks Accept, done. Until they do, NAA sign-in inside the add-in fails for every user in the tenant.

The URL

Same URL for every customer — */organizations/* resolves the tenant from whoever signs in. No substitution needed.

Structured Template

- https://login.microsoftonline.com/organizations/adminconsent?client_id=c2995f31-11e7-4882-b7a7-ef9def0a0266&redirect_uri=https://pivot.claude.ai/auth/callback

Print it. Tell them: open in a browser where a **Global Admin** for their tenant is signed in. They'll see a permissions screen listing what the add-in reads (user profile, extension attributes). After they click **Accept**, they land on a confirmation page — "Admin consent granted, you can close this tab."

Verify

Structured Template

- `az ad sp show --id c2995f31-11e7-4882-b7a7-ef9def0a0266 --query appId -o tsv`

If that returns the same GUID, the service principal exists in their tenant — consent worked. If it errors with “does not exist”, consent didn’t complete.

Outlook — Microsoft Graph consent

Only needed when deploying the Outlook manifest. Separate from *entra_sso* above; required even if *entra_sso* is off.

Claude for Outlook reads mail and calendar through Microsoft Graph. The Graph token stays in the user’s Outlook client and is never sent to the gateway or to Anthropic, so this consent is the same regardless of which cloud serves the model. A Global Admin opens the URL, clicks Accept, done.

Structured Template

- https://login.microsoftonline.com/organizations/v2.0/adminconsent?client_id=c2995f31-11e7-4882-b7a7-ef9def0a0266&scope=https://graph.microsoft.com/Mail.ReadWrite%20https://graph.microsoft.com/Calendars.Read%20https://access&redirect_uri=https://pivot.claude.ai/auth/callback

Without this, every user hits a “Need admin approval” wall the first time Claude tries to read mail.

If their policy forbids consenting to a third-party app: they can register their own single-tenant Entra app with the same delegated Graph permissions (Mail.ReadWrite, Calendars.Read, People.Read, User.Read, offline_access), grant admin consent on it, and pass its client ID as *graph_client_id* when generating the Outlook manifest. Same data flow; approval lives under their app instead of Anthropic's.

3.6 Command: debug.md

File Metadata

Field	Value
description	Diagnose deployment issues (stale config, connect failures, missing add-in)

Debug a Claude Office deployment

You are helping an enterprise admin diagnose why the deployed add-in isn't working right. Start by asking **what's wrong**, then route.

Triage

Ask the admin to describe the symptom. Route by answer:

Symptom	Section
Updated the manifest but users still see old config	Stale config after update
Add-in shows "Connection failed"	Read the error paste
Add-in doesn't appear in Excel/ PowerPoint at all	Add-in not visible
Sign-in popup fails or loops	Admin consent
Need to see the browser console	Opening browser devtools

If they have an error paste from the add-in (the **Copy error details** button on the connect-failed screen), always start there. It carries everything.

Read the error paste

Paste structure:

Structured Template

- Claude for Office connection failed ()
- Build:
- Request:
- :
- ...
- Manifest params:
- :
- ...
- Raw error:

What to check:

- *Request: vs Manifest params:* delta. Keys are the same snake_case names in both blocks, so diff directly. If they differ, the user typed override values into the form. If they match, the manifest values went through unchanged.
 - *Manifest params:* *m* key is the version tag (e.g. *unified-1.0.0.11*). If it's below what you last uploaded, the user is on a stale manifest. Go to [Stale config](#).
 - *Raw error:* is the ground truth. Common patterns:
 - *invalid_client* (401, Google) → wrong *google_client_secret* for that *google_client_id*. Verify in GCP Console → Credentials.
 - *Load failed ()* → network blocked at the WebView layer. Firewall needs to allow that host.
 - *STS AssumeRoleWithWebIdentity failed* → AWS IAM OIDC provider misconfigured or role trust policy wrong.
 - *HTTP 401/403* (gateway) → bad token or gateway rejected the key.
-

Stale config after update

Two caches, two clocks:

Layer	Who holds it	TTL	How to clear
Service	M365 Admin Center → Exchange Online → client	Up to 72h for updates (24h for fresh deploys)	Wait, or redeploy with a fresh “
Client	Office app's Wef folder on each machine	Until app restart, sometimes longer	Delete the folder

Microsoft's own FAQ:

“It can take up to 72 hours for add-in updates, changes from turn on or turn off to reflect for users.
<https://learn.microsoft.com/en-us/microsoft-365/admin/manage/centralized-deployment-faq>”

Confirm what Admin Center is serving

Admin Center silently ignores re-uploads with the same “. If you uploaded a fix without bumping the fourth segment, it never took. Open M365 Admin Center → Integrated apps → your add-in → check the listed version.

Force a client-side refresh

Quit Excel/PowerPoint first, then:

macOS:

Structured Template

- `rm -rf ~/Library/Containers/com.microsoft.Excel/Data/Library/Caches/`
- `rm -rf ~/Library/Containers/com.microsoft.Powerpoint/Data/Library/Caches/`

- `rm -rf ~/Library/Containers/com.microsoft.Excel/Data/Documents/wef`
- `rm -rf ~/Library/Containers/com.microsoft.Powerpoint/Data/Documents/wef`

Windows:

Structured Template

- `rd /s /q "%LOCALAPPDATA%\Microsoft\Office\16.0\Wef"`

Relaunch. If still stale, the service-side cache hasn't caught up. Wait, or use a fresh " (below).

Microsoft's cache-clear doc: <https://learn.microsoft.com/en-us/office/dev/add-ins/testing/clear-cache>

Nuclear option: redeploy with a fresh Id

If 72h is unacceptable, a fresh ' *UUID forces Admin Center and every client to treat it as a brand-new add-in (24h fresh-deploy SLA, usually much faster)*. Edit `manifest.xml`, replace the text inside ' with a new *UUID* (`uuidgen` on *mac/linux*, `[guid]::NewGuid()` in PowerShell), re-upload.

Add-in not visible

- **Not in the ribbon:** Check M365 Admin Center → Integrated apps → your add-in → Users tab. Is the user (or their group) assigned? Nested groups aren't supported.
 - **Shows "My Add-ins" but not the ribbon button:** The manifest's ' *may not include this app*. Check both lists (*top-level and under ')*.
 - **Fresh deploy, been <24h:** Normal. Microsoft's SLA is 24h for first-time deployment visibility.
-

Admin consent

If the user sees a sign-in popup that closes immediately or loops, the tenant hasn't granted admin consent to the Claude app. Run `:consent` to generate the consent URL for a Global Admin to approve. The symptom in error pastes: `user_canceled` in the raw error (the broker maps any unclassifiable close to that).

Opening browser devtools on the add-in

When you need the WebView's console — JS errors, network tab, the add-in's debug logs — you have to attach the host OS's browser devtools. The add-in runs in an embedded WebView with no address bar and no built-in F12, so each OS has its own recipe.

macOS (Safari Web Inspector)

Three gates. **Gate 3 is the one everyone misses.**

1. **Office developer extras** — quit the app first, then: `"bash defaults write com.microsoft.Excel OfficeWebAddinDeveloperExtras -bool true defaults write com.microsoft.Powerpoint OfficeWebAddinDeveloperExtras -bool true defaults write com.microsoft.Word OfficeWebAddinDeveloperExtras -bool true"` Makes right-click → **Inspect Element** appear inside the task pane.

2. **Safari Develop menu** — Safari → Settings → Advanced → check *Show features for web developers*.
3. **macOS Developer Tools allowlist** (Sonoma and later) — System Settings → Privacy & Security → Developer Tools → toggle **Terminal** on. Without this, Safari’s Develop menu shows “*No Inspectable Applications*” even with gates 1 and 2 open.

With the task pane open, either right-click inside it → **Inspect Element**, or go to Safari → Develop → *[your machine name]* → find the add-in host (*pivot.claude.ai* in prod, your configured domain otherwise).

Gotchas:

- **Office updates silently reset gate 1.** If inspection worked last week and doesn’t now, re-run the *defaults write*.
- “*No Inspectable Applications*” = gate 3 missing, or the Office app wasn’t fully quit before *defaults write*. *kill -f “Microsoft Excel”* then relaunch.
- The task pane has to be **open** (not just the app) for it to appear under Safari’s Develop menu.

Windows (Edge DevTools)

Depends on which WebView engine Office is using. Current M365 on Win10/11 with the WebView2 runtime gets Chromium; older perpetual Office or machines without the runtime may still be on IE11/Trident.

WebView2 (Chromium — the common case):

Right-click inside the task pane → **Inspect**. That’s it, no gates. If right-click doesn’t show Inspect, install **Microsoft Edge DevTools Preview** from the Microsoft Store — it lists all attachable WebView2 targets including Office add-ins. Launch it, find the add-in’s URL in the target list, click to attach.

IE11/Trident (legacy Office 2019/2021 perpetual):

Run the IEChooser from an admin PowerShell:

Structured Template

- & “C:\Windows\SysWOW64\F12\IEChooser.exe”

Pick the add-in’s page from the list. If the list is empty, the task pane isn’t open yet — open it first, then refresh IEChooser.

Microsoft’s walkthrough: <https://learn.microsoft.com/en-us/office/dev/add-ins/testing/debug-add-ins-using-devtools-edge-chromium>

3.7 Command: manifest.md

File Metadata

Field	Value
description	Generate the add-in manifest XML with your cloud config baked in

Generate add-in manifest

The script fetches the canonical manifest and appends your config as URL query parameters. The add-in reads them at startup. Outlook uses a separate template because Microsoft’s *MailApp* schema is distinct from the *TaskPaneApp* schema Excel/Word/PowerPoint share, so ask which apps they’re deploying and generate one file per host.

Host arg	Apps	Template
<i>office</i>	Excel, Word, PowerPoint	<i>pivot.claude.ai/manifest.xml</i>
<i>outlook</i>	Outlook (mail + calendar)	<i>pivot.claude.ai/manifest-outlook-3p.xml</i>

Keys by cloud

Prompt only for the keys their cloud path needs. Don’t ask for all eight.

Cloud	Keys
Vertex	<i>gcp_project_id gcp_region google_client_id google_client_secret</i>
Bedrock	<i>aws_role_arn aws_region</i>
Foundry	<i>azure_resource_name azure_api_key</i>
Gateway	<i>gateway_url gateway_token gateway_auth_header gateway_api_format</i>
Gateway (<i>gateway_api_format=vertex</i>)	also <i>gcp_project_id gcp_region</i>

Amazon Bedrock is **not currently supported for the *outlook* host**; the script exits with an error if you pass *aws_** keys with *outlook*.

Outlook — Microsoft Graph

Outlook reads the user’s mailbox and calendar via Microsoft Graph, which requires a one-time tenant-wide admin consent regardless of which cloud serves the model. Run [consent](#) before deploying — otherwise every user hits “Need admin approval” on first open.

If their policy forbids consenting to a third-party app, prompt for *graph_client_id* (their own single-tenant Entra app’s client ID with Mail.ReadWrite, Calendars.Read, People.Read, User.Read, offline_access delegated permissions and admin consent granted). Otherwise leave it unset and the add-in uses Anthropic’s multi-tenant app.

Entra SSO

entra_sso=1 makes the add-in acquire an Entra ID token at startup. Set it when your deployment needs the user’s Microsoft identity — Bedrock uses it as the STS web identity, the bootstrap endpoint uses it as Bearer auth, and per-user attrs ([update-user-attrs](#)) ride inside it as *extn.** claims.

Admin consent is a prerequisite. Without it, every user hits a Microsoft consent dialog on first open. Run [consent](#) first so *entra_sso=1* is silent for your users.

If you don’t need Entra — static gateway config, Vertex with Google OAuth — leave it off. Users won’t see a Microsoft prompt for a setup that doesn’t involve Microsoft.

Bring your own Entra app. By default the token is requested as Anthropic’s multi-tenant app (*c2995f31-...*), so its *aud* claim is that GUID. If your bootstrap endpoint or token-exchange service requires *aud* to match an app registered in *your* tenant, set *graph_client_id*=. Register the app in Entra as a single-tenant **Single-page application** with redirect URI *https://pivot.claude.ai/msal-redirect.html*. You handle consent on your own app — [consent](#) covers the default app only.

Send an access token instead of the ID token. With *graph_client_id* alone the add-in still sends an *ID token* to your bootstrap endpoint — *aud* is your app’s GUID, but there’s no *scp* claim. If your endpoint is a standard OAuth2 protected resource that validates *aud* + *scp*, or an RFC 8693 token-exchange service, set *entra_scope*=*api:///* and the add-in requests an *access token* for that scope instead. The Bearer it sends carries *aud* = your API’s App ID URI and *scp* = the granted scope. In Entra, on your app registration: **Expose an API** (Application ID URI *api:///*), add a scope such as *access_as_user*, and grant the same app delegated permission to it, then grant admin consent for the tenant. In the app manifest, set *accessTokenAcceptedVersion*: 2 so the issued token uses v2.0 claims (*iss* = *login.microsoftonline.com/v2.0*, *azp*, *preferred_username*); leave it unset and you get v1.0 tokens, which your validator may reject. */.default* (requests all consented scopes) also works.

entra_scope requires *graph_client_id* — the build script enforces this. Both are manifest-only: the add-in needs them to initialize NAA *before* it can read extension attrs or call your bootstrap endpoint, so neither can arrive through those layers. Leave *entra_scope* unset and the ID token is sent.

Bootstrap endpoint

bootstrap_url points to an HTTPS endpoint you host. At startup the add-in fetches per-user JSON from it — provider keys, *mcp_servers*, *skills* — and the response overrides manifest values for that user. The URL itself is [interpolated](#) against manifest + attrs before the fetch, so one endpoint can branch on a query param.

See [bootstrap](#) for the request/response contract, JWT validation, and handler scaffolding.

MCP servers

mcp_servers is a JSON array of customer-hosted MCP servers the add-in connects to directly. Each entry is *{url, label, headers?, discover?}* — *headers* present means static auth; absent triggers OAuth discovery. Values interpolate other config keys via *{{gateway_url}}*-style templates.

Setting it here applies one list org-wide; per-user lists belong in [bootstrap](#), which also has the full schema. The value is JSON inside a shell arg — single-quote it:

Structured Template

- `mcp_servers='["url":"{{gateway_url}}/deepwiki/mcp","label":"DeepWiki","headers":{"Authorization":"Bearer {{gateway_token}}"}"]'`

Telemetry

otlp_endpoint routes the add-in’s OpenTelemetry traces to a collector you operate. Set it to the collector’s base HTTPS URL — the add-in appends */v1/traces* and posts OTLP/HTTP. gRPC isn’t supported (the add-in runs in a browser WebView). Leave it unset and no custom collector is configured.

otlp_headers supplies authentication headers for that collector, in the same *key1=value1,key2=value2* format as the standard *OTEL_EXPORTER_OTLP_HEADERS* variable. URL-encode the value in the manifest.

otlp_resource_attributes adds attributes to the OpenTelemetry Resource on every span, in the same

key1=value1,key2=value2 format as the standard *OTEL_RESOURCE_ATTRIBUTES* variable. Use this when your collector requires specific resource attributes for routing or attribution (e.g. *team.name=platform,deployment.environment=prod*). The add-in already sets *service.name*, *service.version*, and *git.sha*; values you provide here are merged on top.

Setting these here applies one collector org-wide; per-user routing belongs in [bootstrap](#) or extension attrs.

Inference headers

inference_headers is a JSON object of extra HTTP headers the add-in attaches to every request it sends to your gateway (*gateway_url*). Use it for accounting or cost-allocation tags your gateway expects — e.g., an internal application ID — so you don’t need a header-injecting proxy in front of it. Applies only when using a gateway; direct cloud connections ignore it.

Structured Template

- `inference_headers='{"x-application-id":"app123"}'`

The add-in treats the values as opaque. *Authorization*, *x-api-key*, *Content-Type*, *Host*, *Content-Length*, *User-Agent*, *Cookie*, and any *anthropic-** / *x-amz-** / *x-goog-** header are reserved and silently dropped — they carry the add-in’s own auth and protocol negotiation.

Setting it here applies one header set org-wide; per-user values belong in [bootstrap](#).

Auto-connect

Default: when all fields for a provider are set, users skip the connection form and land straight in chat. Ask: should they instead see the form first (prefilled, one click)? Yes → *auto_connect=0*.

Allow Claude.ai sign-in

When any enterprise config key is present, users land on the enterprise connection screen and the **Back** button to Claude.ai sign-in is hidden (*allow_lp=0*, the default). Set *allow_lp=1* to keep the **Back** button.

Disabled features

disabled_features is a comma-separated list of feature slugs the admin wants locked for users. Slugs use . form. Currently enforced:

Slug	Effect
<i>skills.authoring</i>	Blocks creating, editing, and uploading skills (create/ update tools, / <i>skillify</i> , . <i>skill</i> upload + drag-drop, skill editing UI). Running admin-provisioned skills is unaffected.

Structured Template

- `disabled_features='skills.authoring'`

Unknown slugs are ignored (forward-compatible). Setting it here applies one policy org-wide; per-user policy belongs in [bootstrap](#) (JSON array) or extension attrs (comma-separated).

Version

M365 Admin Center caches by ‘ + — *re-upload with the same version is silently ignored. After the script writes manifest.xml, ask whether this replaces an existing deployment; if yes, edit ‘ to bump the fourth segment past their last deployed value. First deploy can leave the template’s version as-is.*

Run

Structured Template

- node “\${CLAUDE_PLUGIN_ROOT}/scripts/build-manifest.mjs” office manifest.xml \
- gcp_project_id= \
- gcp_region= \
- auto_connect=0 \
- ...
- # and if they’re also deploying Outlook:
- node “\${CLAUDE_PLUGIN_ROOT}/scripts/build-manifest.mjs” outlook manifest-outlook.xml \
- \
- graph_client_id= # only if NOT using Anthropic’s app via the consent URL

The script validates key names (unknown keys fail hard) and shape-hints values (warns but doesn’t block — their infra may look different).

Validate

Structured Template

- npx –yes office-addin-manifest validate manifest.xml

If validation passes but M365 Admin Center still rejects or ignores the upload, match the symptom below. Edit *manifest.xml* directly, then re-validate.

Symptom	Fix
“An add-in with this ID already exists”	Replace the text inside “ with a fresh UUID. The template carries the marketplace install’s ID.
Re-upload accepted but nothing changes	M365 caches by ID + version. Edit ‘ <i>to a higher fourth segment</i> (e.g. 1.0.0.9 → 1.0.0.10‘) and re-validate.
Only want Excel (not PowerPoint)	Remove ‘ <i>elements for Presentation</i> . **Two parallel lists:** the top-level uses Name=“Presentation”, the one under uses xsi:type=“Presentation” — both must go or the manifest is inconsistent. The xsi:type block is multi-line, delete the whole ...</ Host>‘.
Only want Excel/ PPT, not Outlook	Nothing to remove — Outlook is a separate file. Just don’t generate it.

3.8 Command: setup.md

File Metadata

Field	Value
description	Setup wizard — provision Vertex/ Bedrock/ Foundry/ gateway, admin consent, generate manifest(s)

Claude in Office — Direct Cloud Setup

You are walking an enterprise admin through configuring the Claude Office add-in to call their own cloud instead of Anthropic’s API. The output is a customized *manifest.xml* they deploy via M365 Admin Center.

Before anything else: the setup log lives at `~/Desktop/claude-for-msft-365-install-setup.md` (resolve `~` for their platform). If it exists, read it first — you may be resuming a prior run and can skip completed steps. Start a new `## Run` — section and append each command and its captured output (IDs, URLs) as you go.

Check for Node.js — Steps 4 and 6 shell out to *node* and *npm*. Run *node --version*. If it’s missing, **ask before installing** — it’s their machine. If they say yes, *brew install node* (mac) / *winget install OpenJS.NodeJS* (win) / whatever their package manager is. If no, stop here.

When capturing values from the admin (IDs, URLs, secrets pasted back from a console) — don’t use `AskUserQuestion`. That’s a choice picker; they’re holding a string. Just say “paste the Client ID when you have it” and read it from their next message. Use `AskUserQuestion` only for the actual branch points (gateway vs vertex, per-user vs org-wide).

Step 1 — How does the add-in reach Claude?

Ask this first, because it’s the thing admins get wrong: **do you already run an LLM gateway (LiteLLM, Portkey, Kong, etc.)?**

- **Yes** → *gateway*. Even if the gateway routes to Vertex or Bedrock under the hood — the add-in talks to *your gateway*, not to Google or AWS. You just need the gateway URL.
- **No** → *vertex or bedrock*. The add-in authenticates directly to the cloud provider. Pick where your infra lives.

Path	What it means	Provisioning	Manifest keys
<i>gateway</i>	Add-in → your gateway → (whatever)	None	<i>gateway_url</i> (+ <i>gateway_api_format</i> if not <i>/v1/messages</i>)
<i>vertex</i>	Add-in → Google Vertex AI, Google OAuth client directly		<i>gcp_project_id</i> , <i>gcp_region</i> , <i>google_client_id</i> , <i>google_client_secret</i>
<i>bedrock</i>	Add-in → AWS Bedrock, directly	IAM OIDC provider + role	<i>aws_role_arn</i> , <i>aws_region</i>
<i>foundry</i>	Add-in → Azure AI Foundry, Foundry resource + API key directly		<i>azure_resource_name</i> , <i>azure_api_key</i>

Bedrock and per-user config (bootstrap endpoint or extension attrs) need *entra_sso=1* — the add-in acquires the user's Entra ID token to authenticate those flows. See the Entra SSO section in [manifest](#).

Step 1b — Which Office apps?

Ask: **Excel/Word/PowerPoint, Outlook, or both?** Outlook is a separate manifest and has one extra prerequisite.

If they're deploying Outlook:

- **Bedrock is not currently supported for Outlook.** If they picked *bedrock* in Step 1, Outlook is off the table for now — generate only the *office* manifest.
- **Microsoft Graph admin consent is required.** Run [consent](#) — a Global Admin opens one URL and clicks Accept. Do this before generating the manifest so you can ask whether they're using Anthropic's app (no *graph_client_id* needed) or their own Entra app (capture *graph_client_id*).

Branch to the matching section below.

Vertex AI

1a. Prerequisites

Confirm with the admin:

- GCP project ID (they should know this)
- Region with Claude model quota (typically *us-east5*)

1b. Create the OAuth client

No *gcloud* command exists for this. Open the console link (substitute their project ID), walk them through, they paste back the client ID and secret.

“Open: <https://console.cloud.google.com/apis/credentials?project=> → **Create Credentials** → **OAuth client ID** - Application type: **Web application** - Name: *Claude for Office* - Authorized redirect URI: <https://pivot.claude.ai/auth/callback> → **Create** → copy the **Client ID** and **Client Secret**”

Enable the Vertex API while they're there:

Structured Template

- `gcloud services enable aiplatform.googleapis.com --project=`

Capture: *gcp_project_id*, *gcp_region*, *google_client_id*, *google_client_secret*.

Continue to [Step 3](#). Vertex uses Google OAuth, not Entra, so admin consent isn't needed unless you also opt into per-user config (in which case come back to Step 2 after deciding in Step 3).

Bedrock

1a. Prerequisites

Confirm with the admin:

- AWS account ID and a region with Claude model access (usually *us-east-1*)
- Their Azure tenant ID (from Entra admin center, or *az account show --query tenantId*)
- *aws* CLI configured against the target account

1b. Create OIDC provider + role

Three *aws iam* calls. The trust policy's *aud* condition is the security boundary — only tokens Azure minted for the Claude add-in can assume this role.

Substitute their tenant ID and region:

Structured Template

- TENANT_ID=""
- CLAUDE_APP_ID="c2995f31-11e7-4882-b7a7-ef9def0a0266"
- AWS_REGION="us-east-1"
- ACCOUNT=\$(aws sts get-caller-identity --query Account --output text)
- ISSUER="login.microsoftonline.com/\${TENANT_ID}/v2.0"
- # OIDC identity provider. Thumbprint is required by the API; AWS validates
- # major IdPs via its own trust store, but the param can't be omitted.
- THUMBPRINT=\$(openssl s_client -servername login.microsoftonline.com \
- -connect login.microsoftonline.com:443 /dev/null \
- | openssl x509 -fingerprint -sha1 -noout | cut -d= -f2 | tr -d ':')
- aws iam create-open-id-connect-provider \
- --url "https://\${ISSUER}" \
- --client-id-list "\${CLAUDE_APP_ID}" \
- --thumbprint-list "\${THUMBPRINT}"
- PROVIDER_ARN="arn:aws:iam::\${ACCOUNT}:oidc-provider/\${ISSUER}"
- # Role with trust policy gated on aud.
- aws iam create-role --role-name ClaudeBedrockAccess \
- --assume-role-policy-document '{
- "Version": "2012-10-17",
- "Statement": [{
- "Effect": "Allow",
- "Principal": {"Federated": "\${PROVIDER_ARN}"},

- “Action”: “sts:AssumeRoleWithWebIdentity”,
- “Condition”: {
- “StringEquals”: {“”“\${ISSUER}”:aud”: “”“\${CLAUDE_APP_ID}“”“”}
- }
- }]
- }’
- # Bedrock invoke permissions.
- aws iam put-role-policy --role-name ClaudeBedrockAccess \
- --policy-name BedrockInvoke \
- --policy-document ‘{
- “Version”: “2012-10-17”,
- “Statement”: [{
- “Effect”: “Allow”,
- “Action”: [“bedrock:InvokeModel”, “bedrock:InvokeModelWithResponseStream”],
- “Resource”: [
- “arn:aws:bedrock:::foundation-model/anthropic.”,
- “arn:aws:bedrock:::”“\${ACCOUNT}”:inference-profile/us.anthropic.”
-]
- }]
- }’
- echo “aws_role_arn: arn:aws:iam::\${ACCOUNT}:role/ClaudeBedrockAccess”

If *create-open-id-connect-provider* errors with *EntityAlreadyExists*, a provider for that issuer already exists — that’s fine, the role will trust it. The ARN is deterministic (*arn:aws:iam:::oidc-provider/*).

Capture: *aws_role_arn*, *aws_region*. Add *entra_sso=1* when generating the manifest — Bedrock needs the Entra ID token as the STS web identity.

Continue to [Step 2](#).

Gateway

No provisioning. Ask for the gateway base URL (LiteLLM, Portkey, etc) and the token. If the token varies per user, it goes in [Step 5](#) instead of the manifest.

Capture: *gateway_url*, *gateway_token*.

API format. Ask: does the gateway expose the Anthropic */v1/messages* API, or is it a pass-through to Bedrock (*/model/{id}/invoke...*) or Vertex (*...:rawPredict*)? Almost always Anthropic — LiteLLM/Portkey/Kong default to it, and a unified */v1/messages* route is the point of running a gateway. Only set *gateway_api_format* to *bedrock* or

vertex if the gateway is a thin proxy that preserves the upstream wire format. If *vertex*, also capture *gcp_project_id* (their GCP project) and *gcp_region* (typically *us-east5*) — they’re path segments in the URL the add-in constructs. Point *gateway_url* at the pass-through path, e.g. *https://litellm.acme.com/bedrock* or *.../vertex_ai/v1*.

Auth header scheme. The add-in sends the token as *x-api-key*: by default — this is what LiteLLM, Portkey, and Kong accept out of the box. If your gateway expects *Authorization: Bearer* instead (common for custom/enterprise gateways), set *gateway_auth_header=authorization*. If you’re unsure, run the Step 6 smoke test with *x-api-key* first — a 401 with “no Authorization header” in your gateway logs means you need *authorization*.

Continue to [Step 3](#). Gateway auth is token-based, not Entra, so admin consent isn’t needed unless you also opt into per-user config (in which case come back to Step 2 after deciding in Step 3).

Azure AI Foundry

1a. Prerequisites

Confirm with the admin:

- An Azure AI Foundry resource with at least one Claude model deployed
- The resource name (the subdomain of the endpoint URL — e.g. *contoso-foundry* from *https://contoso-foundry.services.ai.azure.com*)

1b. Get the API key

Open the resource in the Azure Portal, then **Keys and Endpoint** → copy **KEY 1**. The add-in auto-detects which Claude models are deployed in the resource, so no model config is needed here.

Capture: *azure_resource_name*, *azure_api_key*.

Continue to [Step 3](#). Foundry auth is key-based, not Entra, so admin consent isn’t needed unless you also opt into per-user config (in which case come back to Step 2 after deciding in Step 3).

Step 2 — Azure admin consent

Only required when *entra_sso=1* — that is, Bedrock (the Entra token is the STS web identity) or per-user config via extension *attrs/bootstrap*. If neither applies, skip to Step 3.

Read *#{CLAUDE_PLUGIN_ROOT}/commands/consent.md* and follow it.

Step 3 — Decide what’s org-wide vs. per-user

The add-in reads per-user extension attributes first, falls back to manifest params. Any key can live at either layer. So the question is: **of the values captured in Step 1, do any vary per user?**

Ask concretely — don’t make them map it themselves:

- Gateway: is it one URL for everyone? One token, or a token per user?
- Vertex: same project for everyone? Same region, or do some users need a different one for data residency?
- Bedrock: same role for everyone, or team-specific roles?

Answer	Split
Nothing varies	Everything → manifest. Skip Step 5.
Unique per user (e.g. gateway token)	Unique key → Step 5, rest → manifest.

Write the split into the setup log so Step 4 and Step 5 each know their subset.

Step 4 — Generate the manifest

Read `${CLAUDE_PLUGIN_ROOT}/commands/manifest.md` and follow it with the **org-wide** values from Step 3. Generate one file per host from Step 1b:

Structured Template

- node “`${CLAUDE_PLUGIN_ROOT}/scripts/build-manifest.mjs`” office manifest.xml = ...
- node “`${CLAUDE_PLUGIN_ROOT}/scripts/build-manifest.mjs`” outlook manifest-outlook.xml = ...

Then validate each:

Structured Template

- `npx -y office-addin-manifest validate manifest.xml`
- `npx -y office-addin-manifest validate manifest-outlook.xml`

Step 5 — Per-user config

Skip unless Step 3 routed something here. Otherwise pick a mechanism by what you’re carrying:

Carrying	Use	Read
A string or two — token, region	Extension attrs	<code>\${CLAUDE_PLUGIN_ROOT}/commands/update-user-attrs.md</code>
<code>mcp_servers</code> , <code>skills</code> , anything structured	Bootstrap endpoint	<code>\${CLAUDE_PLUGIN_ROOT}/commands/bootstrap.md</code>

Attrs are an *az rest PATCH* per user — less work, but flat strings ≤ 256 chars only. Bootstrap is an HTTPS service you build — more work, no shape limits.

Step 6 — Verify a model is reachable

Before they deploy, confirm at least one of **Claude Sonnet 4.5** or **Claude Opus 4.5** (or newer) actually answers. A manifest that points at an unenabled model deploys fine and then fails silently at first user message.

Gateway: probe with a 1-token request. 200 means it works. 404 means the gateway doesn’t route that model name — try the other, or ask them to check their gateway config. 429 means auth works but no quota on that model — try the other. 401/403 means the token is wrong, which is a Step 1 problem.

Pick the curl that matches *gateway_api_format*. (Windows: swap `/dev/null` for `NUL`. If *gateway_auth_header*=*x-api-key*, swap the auth header line for `-H 'x-api-key: '`.)

gateway_api_format=anthropic (default):

Structured Template

- `curl -s -o /dev/null -w '%{http_code}\n' "/v1/messages" \`
- `-H 'content-type: application/json' -H 'authorization: Bearer ' \`
- `-d '{"model": "claude-sonnet-4-5", "max_tokens": 1, "messages": [{"role": "user", "content": "hi"}]}'`

gateway_api_format=bedrock — model goes in the path, **not** the body:

Structured Template

- `curl -s -o /dev/null -w '%{http_code}\n' \`
- `"/model/anthropic.claude-sonnet-4-5-v1:0/invoke" \`
- `-H 'content-type: application/json' -H 'authorization: Bearer ' \`
- `-d '{"anthropic_version": "bedrock-2023-05-31", "max_tokens": 1, "messages": [{"role": "user", "content": "hi"}]}'`

gateway_api_format=vertex — model, project, and region all go in the path:

Structured Template

- `curl -s -o /dev/null -w '%{http_code}\n' \`
- `"/projects//locations//publishers/anthropic/models/claude-sonnet-4-5:rawPredict" \`
- `-H 'content-type: application/json' -H 'authorization: Bearer ' \`
- `-d '{"anthropic_version": "vertex-2023-10-16", "max_tokens": 1, "messages": [{"role": "user", "content": "hi"}]}'`

Vertex: model enablement is click-ops — the EULA accept has no API. Open the Model Garden page, confirm at least one model shows **Enabled** (not “Request access”) for their region:

“<https://console.cloud.google.com/vertex-ai/publishers/anthropic?project=>”

If it says “Request access”, they click through, accept terms, wait for enable. No API call until they confirm.

Bedrock: same constraint — model access grant has no API. Open the model access page, confirm at least one Claude 4.5+ model shows **Access granted** (not “Available to request”):

“<https://console.aws.amazon.com/bedrock/home?region=#/modelaccess>”

If it says “Available to request”, they request, accept terms, wait for grant (usually minutes, sometimes longer).

Log the verified model name to the setup log. Don’t proceed until you have a 200, a confirmed “Enabled”, or a confirmed “Access granted” — whichever matches their path.

Step 7 — Deploy

Walk them through the upload — there are a few screens, and the user-assignment one is a real decision.

“Open: <https://admin.cloud.microsoft/?#/Settings/IntegratedApps> → **Upload custom apps** - App type: **Office Add-in** - Choose how: **Upload manifest file (.xml) from device** → select *manifest.xml* - It

validates on upload. If it errors here, Step 4's *npx office-addin-manifest validate* should have caught it — re-run that.”

Users screen — the decision point:

- If Step 5 was skipped (nothing varies per user) → **Entire organization** is fine.
- If Step 5 wrote per-user attrs → assign to **Specific users/groups** matching exactly who got PATCHed. Everyone else would open the add-in with no config.
- First deploy? Start with **Just me** or a pilot group, confirm it works, then widen. You can change assignment later without redeploying.

“→ **Accept permissions** → **Finish deployment**”

Propagation to users takes up to 24 hours (usually much faster). The add-in appears under **Home** → **Add-ins** in Excel/Word/PowerPoint once it lands.

Append the final manifest path and the assignment scope to the setup log. Done.

3.9 Command: `update-user-attrs.md`

File Metadata

Field	Value
description	Set per-user config (tokens, region overrides) via Azure AD extension attributes

Per-user config via extension attributes

The attributes are already registered on Anthropic's app (*c2995f31-...*) — you don't create schema, you just write values. The add-in reads *extension_c2995f311e74882b7a7ef9def0a0266_* from the user's ID token.

Requires *entra_sso=1* in the manifest. Without it the add-in never acquires an Entra token, so these attributes are never read — they silently do nothing.

Any of the config keys can be set per-user — the add-in merges per-user attrs over manifest params, so whatever's here wins. All values are 256 chars max.

Key	Per-user use case
<i>gateway_token</i>	Per-user API key (rotation)
<i>gateway_url</i>	Route different teams to different gateways
<i>gateway_api_format</i>	Gateway speaks Bedrock/ Vertex pass-through, not Anthropic / <i>v1/messages</i>
<i>inference_headers</i>	Per-user accounting tag for the gateway (JSON; mind the 256-char cap)
<i>bootstrap_url</i>	Per-user credential-vending endpoint

<code>gcp_project_id</code>	Different teams on different GCP projects
<code>gcp_region</code>	Data-residency override
<code>google_client_id google_client_secret</code>	Different OAuth client per team (uncommon)
<code>aws_role_arn aws_region</code>	Different Bedrock roles by team
<code>otlp_endpoint otlp_headers otlp_resource_attributes</code>	Route telemetry to a team-specific OTEL collector / tag spans with team-level resource attributes

One user

Substitute ‘ with the attribute name from the table. For non-secret keys (regions, project IDs) this is the normal path. For secrets (gateway_token, google_client_secret) the value lands in shell history and this conversation’s transcript — use the bulk CSV path below if that’s a problem.

Structured Template

- `az rest --method PATCH \`
- `--uri "https://graph.microsoft.com/v1.0/users/" \`
- `--body '{ "extension_c2995f3111e74882b7a7ef9def0a0266_": "" }'`

Success is silent — PATCH returns 204 with an empty body. To verify:

Structured Template

- `az rest --method GET --uri "https://graph.microsoft.com/v1.0/users/?$select=extension_c2995f3111e74882b7a7ef9def0a0266_"`

Graph reads are immediately consistent with the write — no lag. To dump every extension attr on a user (without knowing exact key names), use `/beta/`:

Structured Template

- `az rest --method GET --uri "https://graph.microsoft.com/beta/users/" | jq 'to_entries | map(select(.key | startswith("extension")))| from_entries'`

Bulk (CSV, values never enter this chat)

Have the admin prepare `users.csv`. First column is UPN; remaining column headers are the attribute keys. Empty cells skip that attr for that user.

Structured Template

- `upn,gateway_token,gcp_region`
- `alice@acme.com,sk-live-aaa,`
- `bob@acme.com,sk-live-bbb,europe-west4`
- `carol@acme.com,,europe-west4`

macOS/Linux — write this to `apply.sh` next to their CSV (the `read -a` array syntax is bash-only; a bare paste into `zsh` breaks). They review it, then run `bash apply.sh`. You only see ✓/✗ — don’t `cat` either file.

Structured Template

- #!/bin/bash
- EXT=extension_c2995f3111e74882b7a7ef9def0a0266_
- {
- IFS=, read -ra keys
- while IFS=, read -ra vals; do
- upn="\${vals[0]}"
- for i in "\${!keys[@]"; do
- ["\$i" -eq 0] || [-z "\${vals[\$i]}"] && continue
- az rest --method PATCH --uri "https://graph.microsoft.com/v1.0/users/\$upn" \
- --body "{ \"\${EXT}\${keys[\$i]}\" : \"\${vals[\$i]}\" }" \
- && echo "✓ \$upn \${keys[\$i]}" || echo "✗ \$upn \${keys[\$i]}"
- done
- done
- } < users.csv

Windows — write this to *apply.ps1* next to their CSV. *Import-Csv* reads the header as the schema directly; they run *.\apply.ps1* in PowerShell.

Structured Template

- \$EXT = 'extension_c2995f3111e74882b7a7ef9def0a0266_'
- Import-Csv users.csv | ForEach-Object {
- \$upn = \$_.upn
- \$_.PSObject.Properties | Where-Object { \$_.Name -ne 'upn' -and \$_.Value } | ForEach-Object {
- \$body = @{"\$EXT\$(\$_.Name)" = \$_.Value } | ConvertTo-Json -Compress
- az rest --method PATCH --uri "https://graph.microsoft.com/v1.0/users/\$upn" --body \$body
- if (\$?) { "OK \$upn \$((\$_.Name))" } else { "FAIL \$upn \$((\$_.Name))" }
- }
- }

Report the ✓ and ✗ counts. 404 means the UPN is wrong; 403 means *az login* lacks *User.ReadWrite.All* — they need to re-consent or use an admin account.

Propagation delay

Graph writes succeed immediately, but the add-in reads these via the user's ID token at NAA sign-in — and Azure's STS caches token claims. Expect **up to an hour** before the new value appears for a given user. If they open the add-in right after the PATCH and it behaves as if unconfigured, that's the cache, not a failure. Tell them

to wait and retry; quitting the Office app fully (not just closing the taskpane) forces a fresh NAA token on next launch.

3.10 Directory: examples

3.11 Script File: examples/python-bootstrap/app.py

Script Summary

Signal	Details
Imports	re, time, jwt , config import , fastapi import FastAPI, Header, HTTPException, fastapi.middleware.cors import CORSMiddleware, jwt import PyJWKClient
Functions	parse_ app, resolve, validate, bootstrap
Entry point	Defines an executable main entry point
Process execution	Invokes subprocesses or external commands

Normalized Script Content

- `"""`
- Claude in Office — Bootstrap endpoint reference implementation.
- The Office add-in calls GET /bootstrap with the user's Entra ID token.
- This server validates the token, decides which skills and MCP servers
- that employee is allowed to use, and returns them.
- All customer-editable settings live in config.py — edit that, not this file.
- `"""`
- `import re`
- `import time`
- `import jwt # PyJWT`
- `from config import (`
- `AUDIENCE,`
- `DEV_JWKS_PATH,`
- `HOST,`
- `ISSUER,`
- `JWKS_URL,`
- `MCP_SERVERS,`
- `PORT,`

- RULES,
- SKILLS,
-)
- from fastapi import FastAPI, Header, HTTPException
- from fastapi.middleware.cors import CORSMiddleware
- from jwt import PyJWKClient
- _UA_RE = re.compile(r“^claude-(word|excel|powerpoint)/”, re.I)
- def parse_app(user_agent: str | None) -> str:
- m = _UA_RE.match(user_agent or “”)
- return m.group(1).lower() if m else “”
- def resolve(oid: str, groups: set[str], app: str) -> dict:
- for r in RULES:
- w = r[“when”]
- if “user” in w and w[“user”] != oid:
- continue
- if “group” in w and w[“group”] not in groups:
- continue
- if “app” in w and w[“app”] != app:
- continue
- return {
- “skills”: [{“name”: n, **SKILLS[n]} for n in r.get(“skills”, [])],
- “mcp_servers”: [MCP_SERVERS[n] for n in r.get(“mcp_servers”, [])],
- }
- return {“skills”: [], “mcp_servers”: []}
- # Token validation
- _jwks = PyJWKClient(JWKS_URL) if not DEV_JWKS_PATH else None
- def validate(auth_header: str) -> dict:
- if not auth_header or not auth_header.startswith(“Bearer ”):
- raise HTTPException(401, “Missing bearer token”)
- token = auth_header.removeprefix(“Bearer ”).strip()
- if DEV_JWKS_PATH:
- import json

- with open(DEV_JWKS_PATH) as f:
- key = jwt.PyJWK(json.load(f)[“keys”][0]).key
- else:
- key = _jwks.get_signing_key_from_jwt(token).key
- try:
- return jwt.decode(token, key, algorithms=[“RS256”], audience=AUDIENCE, issuer=ISSUER)
- except jwt.InvalidTokenError as e:
- raise HTTPException(401, f“Invalid token: {e}”)
- # HTTP
- app = FastAPI()
- app.add_middleware(
- CORSMiddleware,
- allow_origins=[“https://pivot.claude.ai”],
- allow_methods=[“GET”],
- allow_headers=[“*”], # FastAPI reflects the preflight’s requested headers
-)
- @app.get(“/bootstrap”)
- def bootstrap(
- authorization: str = Header(None),
- x_claude_user_agent: str = Header(None),
-):
 - claims = validate(authorization)
 - oid = claims.get(“oid”, “”)
 - # NOTE: We assume group membership arrives in the token’s groups claim.
 - # If your tenant doesn’t emit it (or you prefer your own RBAC), replace
 - # this line with a lookup against your IdP / HRIS — e.g.
 - # groups = fetch_groups_from_graph(oid) or your_iam.groups_for(email)
 - groups = set(claims.get(“groups”, []))
 - config = resolve(oid, groups, parse_app(x_claude_user_agent))
 - return {**config, “bootstrap_expires_at”: int(time.time()) + 3600}
 - if __name__ == “__main__”:
 - import uvicorn

- `uvicorn.run(app, host=HOST, port=PORT)`

3.12 Script File: `examples/python-bootstrap/config.py`

Script Summary

Signal	Details
Imports	<code>base64</code> , <code>os</code>
Functions	<code>b64</code>

Normalized Script Content

- `"""`
- Edit this file to configure your bootstrap server. `app.py` should not need changes.
- `"""`
- `import base64`
- `import os`
- `# Config`
- `TENANT_ID = os.environ["TENANT_ID"] # your Entra tenant`
- `AUDIENCE = "c2995f31-11e7-4882-b7a7-ef9def0a0266" # Claude in Office add-in app ID`
- `ISSUER = f"https://login.microsoftonline.com/{TENANT_ID}/v2.0"`
- `JWKS_URL = f"https://login.microsoftonline.com/{TENANT_ID}/discovery/v2.0/keys"`
- `HOST = os.getenv("HOST", "127.0.0.1")`
- `PORT = int(os.getenv("PORT", "8080"))`
- `# Local-dev override: point at a self-issued JWKS instead of Entra.`
- `# Signature verification still runs. Refuses to start on non-loopback.`
- `DEV_JWKS_PATH = os.getenv("DEV_JWKS_PATH")`
- `if DEV_JWKS_PATH and HOST != "127.0.0.1":`
- `raise SystemExit("DEV_JWKS_PATH may only be used when HOST=127.0.0.1")`
- `# Catalog: every skill / MCP server you might hand out`
- `def b64(s: str) -> str:`
- `return base64.b64encode(s.encode()).decode()`
- `SKILLS = {`
- `"deal-memo": {`
- `"description": "Draft a deal memo from a term sheet",`

- “url”: “https://your-bucket.s3.amazonaws.com/skills/deal-memo.zip?X-Amz-Signature=...”
- },
- “compliance-check”: {
- “description”: “Review a document for compliance issues”,
- “content”: b64(“# Compliance check\n\nReview the document for regulatory red flags...”)
- },
- “risk-dashboard”: {
- “description”: “Summarize positions from the risk dashboard”,
- “url”: “https://your-bucket.s3.amazonaws.com/skills/risk.zip?X-Amz-Signature=...”
- },
- }
- MCP_SERVERS = {
- “linear”: {“url”: “https://mcp.linear.app/sse”, “label”: “Linear”},
- “risk-api”: {
- “url”: “https://internal.example.com/mcp/risk”,
- “label”: “Risk Dashboard”,
- “headers”: {“Authorization”: “Bearer {{gateway_token}}”},
- },
- }
- # RBAC: first matching rule wins
- # when conditions (all must match):
- # group — value from the Entra token’s groups claim
- # user — Entra user oid
- # app — Office host: “word” | “excel” | “powerpoint”
- # In production, group/user values are GUIDs — replace the names below
- # with real Object IDs from Entra admin center.
- RULES = [
- {“when”: {“app”: “word”, “group”: “investment-banking”},
- “skills”: [“deal-memo”, “compliance-check”, “mcp_servers”: [“linear”]},
- {“when”: {“group”: “risk”},
- “skills”: [“risk-dashboard”, “compliance-check”, “mcp_servers”: [“risk-api”]},
- {“when”: {“user”: “alice”},

- “skills”: [“deal-memo”], “mcp_servers”: [],
- {“when”: {}, “skills”: [“compliance-check”], “mcp_servers”: []}, # default
-]

3.13 Script File: examples/python-bootstrap/get_tenant_id.py

Script Summary

Signal	Details
Imports	json, subprocess, sys, urllib.request
Functions	from_domain, from_az_cli
Entry point	Defines an executable main entry point
Process execution	Invokes subprocesses or external commands

Normalized Script Content

- `#!/usr/bin/env python3`
- `"""`
- Print your Entra (Azure AD) tenant ID.
- Usage:
- `python get_tenant_id.py alice@yourcompany.com`
- `python get_tenant_id.py yourcompany.com`
- `python get_tenant_id.py # tries az account show if Azure CLI is logged in`
- Set the result as TENANT_ID before running app.py.
- `"""`
- `import json`
- `import subprocess`
- `import sys`
- `import urllib.request`
- `def from_domain(domain: str) -> str:`
- `# Entra publishes per-tenant OIDC metadata at this well-known URL.`
- `# The issuer field is https://login.microsoftonline.com/v2.0`
- `url = f'https://login.microsoftonline.com/{domain}/v2.0/.well-known/openid-configuration'`
- `with urllib.request.urlopen(url, timeout=5) as r:`
- `issuer = json.load(r)["issuer"]`

- `return issuer.rstrip("/").split("/")[-2]`
 - `def from_az_cli() -> str:`
 - `out = subprocess.run(`
 - `["az", "account", "show", "--query", "tenantId", "-o", "tsv"],`
 - `capture_output=True, text=True, check=True,`
 - `)`
 - `return out.stdout.strip()`
 - `if __name__ == "__main__":`
 - `if len(sys.argv) > 1:`
 - `arg = sys.argv[1]`
 - `domain = arg.split("@", 1)[1] if "@" in arg else arg`
 - `print(from_domain(domain))`
 - `else:`
 - `try:`
 - `print(from_az_cli())`
 - `except Exception as e:`
 - `sys.exit(f"Pass an email/domain, or run az login first ({e})")`
-

3.14 Script File: `examples/python-bootstrap/mint_dev_token.py`

Script Summary

Signal	Details
Imports	<code>argparse, json, os, time, base64, jwt, cryptography.hazmat.primitives import serialization, cryptography.hazmat.primitives.asymmetric import rsa</code>
Functions	<code>b64u</code>
CLI	Exposes command-line argument parsing

Normalized Script Content

- `"""`
- Mint a self-signed dev token for local testing of `app.py`.
- First run generates `dev_private.pem` + `dev_jwks.json` in the current directory.
- Subsequent runs reuse them.
- `python mint_dev_token.py --oid alice --group --group`

```

• """
• import argparse, json, os, time, base64
• import jwt
• from cryptography.hazmat.primitives import serialization
• from cryptography.hazmat.primitives.asymmetric import rsa
• PRIV = "dev_private.pem"
• JWKS = "dev_jwks.json"
• AUDIENCE = "c2995f31-11e7-4882-b7a7-ef9def0a0266"
• TENANT_ID = os.getenv("TENANT_ID", "dev-tenant")
• ISSUER = f"https://login.microsoftonline.com/{TENANT_ID}/v2.0"
• def b64u(n: int) -> str:
•     raw = n.to_bytes((n.bit_length() + 7) // 8, "big")
•     return base64.urlsafe_b64encode(raw).rstrip(b"=").decode()
• if not os.path.exists(PRIV):
•     key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
•     with open(PRIV, "wb") as f:
•         f.write(key.private_bytes(
•             serialization.Encoding.PEM,
•             serialization.PrivateFormat.PKCS8,
•             serialization.NoEncryption(),
•         ))
•     nums = key.public_key().public_numbers()
•     jwk = {"kty": "RSA", "kid": "dev", "alg": "RS256", "use": "sig",
•           "n": b64u(nums.n), "e": b64u(nums.e)}
•     with open(JWKS, "w") as f:
•         json.dump({"keys": [jwk]}, f, indent=2)
•     ap = argparse.ArgumentParser()
•     ap.add_argument("--oid", default="alice")
•     ap.add_argument("--group", action="append", default=[])
•     args = ap.parse_args()
•     with open(PRIV, "rb") as f:
•         priv = serialization.load_pem_private_key(f.read(), password=None)

```

- `claims = {"aud": AUDIENCE, "iss": ISSUER, "oid": args.oid,`
 - `"groups": args.group, "exp": int(time.time()) + 3600}`
 - `print(jwt.encode(claims, priv, algorithm="RS256", headers={"kid": "dev"}))`
-

3.15 File: `examples/python-bootstrap/README.md`

Bootstrap endpoint — Python reference

A minimal FastAPI implementation of the Claude in Office */bootstrap* endpoint. It validates the caller's Entra ID token and returns per-employee *skills* and *mcp_servers* based on a simple first-match RBAC table.

Run against your real Entra tenant

Structured Template

- `pip install -r requirements.txt`
- # Find your tenant ID:
- `python get_tenant_id.py you@yourcompany.com`
- `export TENANT_ID=`
- `python app.py`

Run locally with a fake token

Structured Template

- `pip install -r requirements.txt`
- `export TENANT_ID=dev-tenant`
- `TOKEN=$(python mint_dev_token.py --oid alice --group investment-banking)`
- `DEV_JWKS_PATH=dev_jwks.json python app.py &`
- `curl -H "Authorization: Bearer $TOKEN" \`
- `-H "X-Claude-User-Agent: claude-word/1.0.0" \`
- `http://127.0.0.1:8080/bootstrap`

Customize

Everything you need to change lives in *config.py* — *app.py* should not need edits.

- Edit *SKILLS* and *MCP_SERVERS* — the full catalog you can hand out.
- Edit *RULES* — first matching rule wins; the empty *when: {}* at the bottom is the default.
- Replace the placeholder group/user names in *RULES* with your real Entra Object IDs (GUIDs).

- Group membership is read from the token's *groups* claim. If your tenant doesn't emit it, swap the *groups* = ... line in *app.py* for a lookup against your internal directory.
- Rules can be scoped per Office host with "*app*": "*word*" | "*excel*" | "*powerpoint*", parsed from the *X-Claude-User-Agent* header the add-in sends.
- The *groups* claim is **not** in Entra tokens by default. Enable it under *App registration* → *Token configuration* → *Add groups claim* for your app.
- Swap the in-memory *RULES* for your real source of truth (DB, config service, etc.).

Security

DEV_JWKS_PATH lets the server trust a self-issued signing key instead of Microsoft's. It refuses to start unless bound to *127.0.0.1*. **Never** set it in a deployed environment.

3.16 File: examples/python-bootstrap/requirements.txt

Structured File Content

- fastapi
 - uvicorn
 - PyJWT[crypto]
-

3.17 Directory: README.md

3.18 File: README.md

Claude for Office — Direct Cloud Setup

Admin tooling for configuring the Claude Office add-in to call your own cloud (Vertex AI, Bedrock, or an LLM gateway) instead of Anthropic's API.

Install

Structured Template

- claude plugin marketplace add anthropics/financial-services-plugins
- claude plugin install claude-for-msft-365-install@financial-services-plugins

Then inside the session: */claude-for-msft-365-install:setup*

Commands

Command	What it does
---------	--------------

<i>/ claude-for-msft-365-install:setup</i>	Interactive wizard — provisions cloud resources, admin consent, writes manifest
<i>/ claude-for-msft-365-install:manifest</i>	Generate the customized add-in manifest XML
<i>/ claude-for-msft-365-install:consent</i>	Azure admin consent URL for the add-in's app registration
<i>/ claude-for-msft-365-install:update-user-attrs</i>	Write per-user config via Microsoft Graph extension attributes
<i>/ claude-for-msft-365-install.bootstrap</i>	Build the bootstrap endpoint — per-user MCP servers, skills, dynamic config

3.19 Directory: scripts

3.20 Script File: scripts/build-manifest.mjs

Normalized Script Content

- `#!/usr/bin/env node`
- `// Fetches the canonical add-in manifest and writes a customized copy with your`
- `// org's config baked into the taskpane URL as query parameters.`
- `//`
- `// Usage: node build-manifest.mjs key=value [key=value ...]`
- `// Example: node build-manifest.mjs office acme.xml gcp_project_id=acme gcp_region=us-east5`
- `import { writeFileSync } from "node:fs";`
- `const MANIFESTS = {`
- `office: "https://pivot.claude.ai/manifest.xml", // Excel + Word + PowerPoint (TaskPaneApp)`
- `outlook: "https://pivot.claude.ai/manifest-outlook-3p.xml", // Outlook (MailApp — separate schema)`
- `};`
- `// Every URL slot Office reads from must carry the same params. Outlook's MailApp`
- `// schema repeats Taskpane.Url across V1_0 and V1_1 VersionOverrides, hence /g.`
- `const URL_SLOTS = [/(<SourceLocation\s+DefaultValue="(("[^"]+)"|"/g, /(id="Taskpane\.Url"\s+Default-`
`Value=")(("[^"]+)"|"/g];`
- `// Recognized config keys. pattern is a shape hint — mismatches warn but don't block`
- `// (your infra may look different). secret keys warn louder: the manifest is an`
- `// org-wide file and its URL can land in deploy logs; per-user secrets typically go`
- `// in Azure extension attributes instead.`
- `const KEYS = {`
- `gcp_project_id: { pattern: /^[a-z][a-z0-9-]{4,28}[a-z0-9]$/, hint: "GCP project ID" },`

- gcp_region: { pattern: /./, hint: “GCP region, e.g. us-east5 or global” },
- google_client_id: { pattern: /\.apps\.googleusercontent\.com\$/, hint: “OAuth 2.0 client ID” },
- google_client_secret: { pattern: /^GOCSXPX-/, hint: “OAuth 2.0 client secret” },
- aws_role_arn: {
 - pattern: /^arn:aws:iam::\d{12}:role\$/,
 - hint: “e.g. arn:aws:iam::123456789012:role/ClaudeBedrockAccess”,
- },
- aws_region: { pattern: /^[a-z]{2}-[a-z]+\d+\$/, hint: “e.g. us-east-1” },
- azure_resource_name: {
 - pattern: /^[a-z0-9][a-z0-9-]{1,62}\$/,
 - hint: “Azure AI Foundry resource name — the subdomain of your endpoint URL, e.g. ‘contoso-foundry’ from https://contoso-foundry.services.ai.azure.com”,
- },
- azure_api_key: {
 - pattern: /^[A-Za-z0-9]{20,}\$/,
 - hint: “From Azure Portal → your Foundry resource → Keys and Endpoint → KEY 1”,
- },
- graph_client_id: {
 - pattern: /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}\$/i,
 - hint: “Entra app (client) ID for Microsoft Graph — Outlook only; omit to use Anthropic’s multi-tenant app via the admin consent URL”,
- },
- gateway_url: { pattern: /^https:\//, hint: “HTTPS base URL” },
- gateway_token: { pattern: /./, hint: “gateway API key”, secret: true },
- gateway_auth_header: { pattern: /^(x-api-key|authorization)\$/i, hint: “auth header scheme (default: x-api-key)” },
- gateway_api_format: { pattern: /^(anthropic|bedrock|vertex)\$/i, hint: “anthropic | bedrock | vertex” },
- mcp_servers: { pattern: /^[.]*\$/, hint: “JSON array of {url, label, headers?, discover?}” },
- inference_headers: { pattern: /^\{.*\}\$/, hint: “JSON object of extra headers to attach to every model request” },
- bootstrap_url: { pattern: /^https:\//, hint: “HTTPS endpoint returning per-user config” },
- otel_endpoint: { pattern: /^https:\//, hint: “OTLP/HTTP traces collector URL” },
- otel_headers: { pattern: /./, hint: “comma-separated k=v pairs for the OTLP exporter” },
- otel_resource_attributes: {

- pattern: `/^([^\s,]+=[^\s,])?([^\s,]+=[^\s,])?$/`,
- hint: “comma-separated k=v pairs added to the OTEL Resource (same format as OTEL_RESOURCE_ATTRIBUTES)”,
- },
- auto_connect: { pattern: `/^[01]$/`, hint: “0 shows form, 1 (or omit) auto-connects” },
- entra_sso: { pattern: `/^[01]$/`, hint: “1 enables Entra SSO (required for aws_role_arn)” },
- graph_client_id: {
- pattern: `/^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i`,
- hint: “your Entra app registration’s Application (client) ID — overrides the default multi-tenant app”,
- },
- entra_scope: {
- pattern: `/^(api|https):\\S+\\[\\w.-]+$/i`,
- hint: “scope URI for your Entra-protected API, e.g. api:///default — requires graph_client_id”,
- },
- allow_1p: {
- pattern: `/^[01]$/`,
- hint: “1 allows Claude.ai OAuth alongside 3P (default: locked when other keys present)”,
- },
- disabled_features: {
- pattern: `/^[\\w.]+(,[\\w.]+)*$/`,
- hint: “comma-separated feature slugs to lock for users, e.g. skills.authoring”,
- },
- };
- const NEEDS_ENTRA = [“aws_role_arn”, “graph_client_id”, “entra_scope”];
- async function main() {
- const [host, out, ...pairs] = process.argv.slice(2);
- const manifestUrl = process.env.MANIFEST_URL || MANIFESTS[host];
- if (!manifestUrl || !out || pairs.length === 0) {
- console.error(“Usage: node build-manifest.mjs key=value [key=value ...]”);
- console.error(Keys: \${Object.keys(KEYS).join(“ ”)});
- process.exit(1);
- }

- if (host === “outlook” && pairs.some((p) => p.startsWith(“aws_”))) {
- console.error(“error: Amazon Bedrock (aws_role_arn/aws_region) is not currently supported for Outlook”);
- process.exit(1);
- }
- if (host !== “outlook” && pairs.some((p) => p.startsWith(“graph_client_id=”))) {
- console.warn(“note: graph_client_id only applies to Outlook; it has no effect in the office manifest”);
- }
- const params = new URLSearchParams();
- for (const p of pairs) {
- const eq = p.indexOf(“=”);
- if (eq < 1) throw new Error(bad arg: \${p} (expected key=value));
- const [k, v] = [p.slice(0, eq).trim(), p.slice(eq + 1).trim()];
- const spec = KEYS[k];
- if (!spec) throw new Error(unknown key: \${k}\n valid: \${Object.keys(KEYS).join(“ , ”)});
- if (!v) throw new Error(empty value for \${k});
- if (!spec.pattern.test(v)) console.warn(warn: \${k}=\${v} — expected \${spec.hint});
- if (spec.secret) {
- console.warn(
- note: \${k} in the manifest applies to every user. If it varies per user, set it via update-user-attrs instead.,
-);
- }
- params.set(k, v);
- }
- const needsEntra = NEEDS_ENTRA.find((k) => params.has(k));
- if (needsEntra && params.get(“entra_sso”) !== “1”) {
- throw new Error(`\${needsEntra} requires entra_sso=1 (the add-in needs an Entra token to use it));
- }
- if (params.has(“entra_scope”) && !params.has(“graph_client_id”)) {
- throw new Error(“entra_scope requires graph_client_id (the scope is requested as your own Entra app, not the default)”);
- }
- // URLSearchParams joins with &; XML attribute values need it escaped.

- `const qs = params.toString().replaceAll("&", "&");`
 - `const res = await fetch(manifestUrl);`
 - `if (!res.ok) throw new Error(fetch ${manifestUrl}: ${res.status} ${res.statusText});`
 - `let xml = await res.text();`
 - `for (const slot of URL_SLOTS) {`
 - `slot.lastIndex = 0;`
 - `if (!slot.test(xml)) throw new Error(manifest missing expected URL slot: ${slot.source});`
 - `slot.lastIndex = 0;`
 - `// The template URL already carries ?m= — append with & not a second ?`
 - `xml = xml.replace(slot, (_, pre, url, post) => pre + url + (url.includes("?") ? "&" : "?") + qs + post);`
 - `}`
 - `writeFileSync(out, xml);`
 - `console.log(Wrote ${out} (${host}, params: ${params}));`
 - `}`
 - `main().catch((err) => {`
 - `console.error(err.message || err);`
 - `process.exit(1);`
 - `});`
-